

[Remote Controlled Acquisition and Launch Robot] — Post-Mortem & As-Built Analysis

PROJECT:	<i>[Laboratorium Teknik Fisika II - Remote Controlled Acquisition and Launch Robot using ESP32 microcontroller]</i>
AUTHOR:	<i>[Zalix Razera Resmana — Electrical & Documentation Lead]</i>
DATE:	<i>[June 2026]</i>
SYSTEM STATUS:	<i>[Concluded / Archive]</i>

1. Executive Summary

This document is a post-mortem analysis of the Remote Controlled Acquisition and Launch Robot, the final project for the 2024 Engineering Physics Laboratory II course. Built by a 6–7 person team across mechanical, electrical, and software systems, the robot was evaluated through a standardized test and 1v1 competition format and failed during the competition. This analysis addresses the underlying problems and constraints that led to that outcome. The findings point to breakdowns in configuration control, cross-team communication, and time management under deadline pressure. Insights intended to help future teams avoid the same failure points.

2. Original System Design

This section establishes the intended architecture and documented boundaries prior to deployment, testing, or the incident under review.

2.1 Architecture & Subsystems

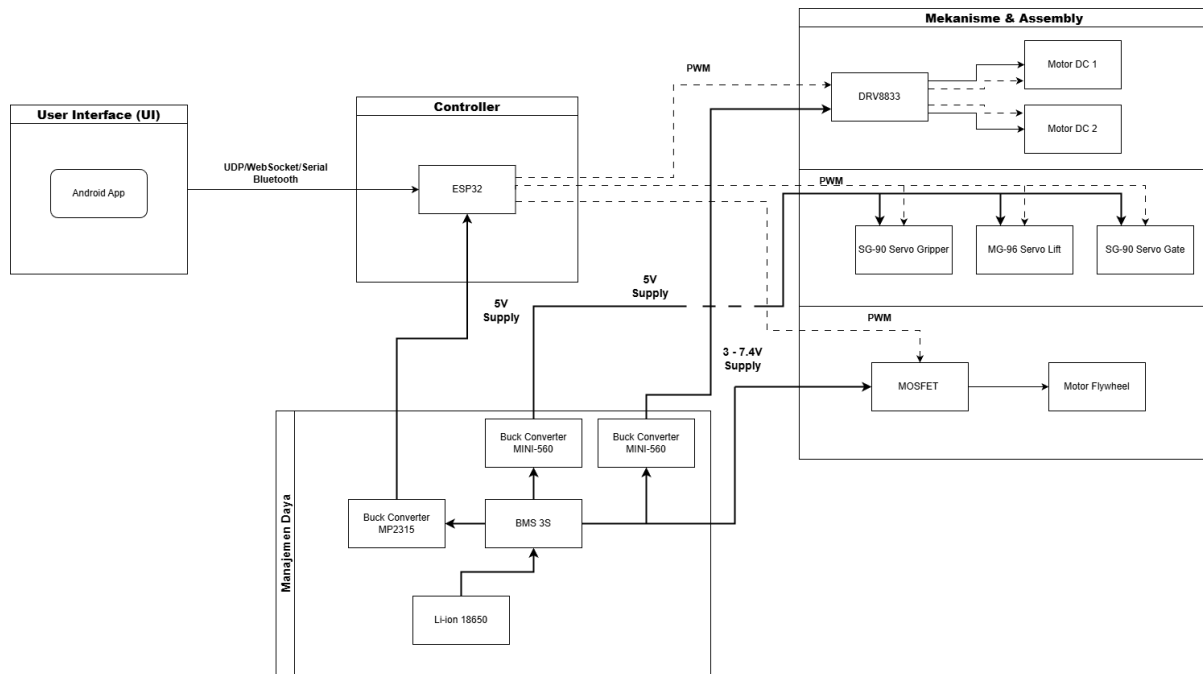
Primary Function / Purpose: The robot is expected to acquire a 4mm foam ball, then launch it toward can target 40–50cm away.

- Acquisition / Intake Mechanism: a parallel gripper paired with a lifting servo, used to grab the foam ball and raise it into position for launch.
- Launch Mechanism: a single flywheel launcher, using a compression ratio between the ball and the wheel to control launch velocity.
- Drivetrain / Chassis: a three-wheel 2WD configuration with an omnidirectional ball caster as the third contact point, manually controlled via an Android app built in Kodular.
- Control Software: an ESP32 as the main controller, communicating with the Android app over UDP (via Kodular's UDP extension) for low-latency manual control.
- Power System: a 3S configuration of 18650 Li-ion cells (2500mAh each), with three separate voltage regulators; an MP2315 dedicated to the ESP32, one MINI-560 buck converter supplying the servos, and a second MINI-560 buck converter supplying the flywheel motor.

2.2 Initial Documentation Boundaries

- **Baseline Reference Document:** A complete functional block diagram (created in Draw.io) plus a Bill of Materials, covering the full signal and power path from user input to actuation.
- **Design Topology / Standard:** The diagram enforced a layered architecture (User Interface → Controller → Power Management → Mechanism/Assembly), with explicit visual separation between PWM/signal lines (dashed) and power supply lines (solid), and specified exact voltage regulation stages for each load type (logic, servos, flywheel motor).

Figure 2.2.1 — Baseline System Architecture Diagram



3. Configuration Drift & Integration Challenges (As-Built State)

During deployment/operation, the actual implementation diverged from the documented baseline. This section captures specific unmapped changes and their cascading consequences.

Drift Model: Baseline Draw.io Architecture → Undocumented Field Fixes (Under Time Pressure) → Unlogged As-Built State → Competition-Day Failure

CASE STUDY 1: Drivetrain Current Inrush & DRV8833 Safety Trip

What changed: The drivetrain's actual load (robot weight) was underestimated during design, so when the drive motor fought initial inertia to move, current draw exceeded the DRV8833 driver's ~1.5A peak rating, triggering its built-in safety shutdown and stalling the wheel. The fix was adding 100μF capacitor directly to the DRV8833's power pins to absorb the current inrush.

System impact: Cost significant debugging time tracing the stall back to the driver's current limit before the root cause was identified.

CASE STUDY 2: Flywheel Compression Ratio Escalation & Cascading Mechanical Instability

What changed: The flywheel-to-ball compression ratio was originally planned at 4–5%, but rose to 10% in practice. This forced a flywheel motor swap to accommodate the new requirement, which in turn left the connection shaft unsupported by the original mount, requiring a custom wood shaft as a workaround.

System impact: The custom shaft introduced rotational instability and vibration, causing inconsistent shots, and ultimately led to a flywheel jam during the first competition duel, a direct contributor to the competition failure.

CASE STUDY 3: Gripper Arm Mechanical Failure (Match 2)

What changed: The gripper arm was attached only via a geared connection point, with no locking mechanism to secure it in place. During the second match, after the drivetrain and flywheel jam issues had already been patched, the geared joint failed under operational load and one arm detached.

System impact: With the gripper arm dropped, the robot lost the ability to acquire the ball entirely, functionally ending its participation in the match. This was the decisive failure that led to the competition loss.

3.1 Workflow / Process Lifecycle Failures

All three hardware fixes documented in this section, the DRV8833 capacitor addition, the flywheel motor swap and custom shaft, and the underlying gripper design gap, were made under acute time pressure, with final mechanical assembly completed less than 24 hours before the competition. None of these changes were fed back into the Draw.io architecture diagram or Bill of Materials. As a result, the documented baseline and the actual as-built robot diverged significantly by competition day, with no record of what had changed, why, or what risks remained unresolved (such as the gripper's missing locking mechanism). This gap between "what was designed" and "what was actually built" meant the team was troubleshooting a robot that no longer matched its own reference documents.

4. Key Takeaways & Systems Engineering Insights

The challenges encountered on this platform provide concrete, actionable parameters for future engineering work.

Strict Configuration Control Enforcement:

From the outset, cross-field communication was treated as a personal discipline rather than an enforced process, changes to any circuit or subsystem were meant to be communicated directly to the people impacted. However, this depended on individual initiative rather than a hard gate. The clear lesson is that no fix should be implemented before the corresponding diagram or BOM entry is updated, and any major change should be validated through a brief team review before implementation, not to slow the team down, but to confirm the change is actually the correct solution and that the rest of the system can accommodate it. Had this gate existed, the DRV8833 fix, the flywheel shaft swap, and the gripper's design gap would have surfaced as team-visible decisions instead of last-minute, undocumented patches.

Documentation as a Structural Boundary:

A dedicated technical specification and decision log was maintained and shared across the team from the start, specifically designed to be updated when a change didn't translate cleanly to a given field. In practice, however, the team treated the specification as a fixed template to follow rather than a living document to actively maintain updates were rarely initiated even when a subsystem's real-world constraints diverged from what was written. This reveals a deeper issue than a missing process: documentation only functions as a structural boundary between disciplines if every contributor treats it as their responsibility to keep current, not just a reference to comply with. When the electrical, mechanical, and software teams stopped feeding real changes back into the shared spec, it silently stopped reflecting reality and nobody noticed until the gap surfaced as a failure on the competition floor.

Design for Diagnostics:

On subsequent projects (e.g., the ESP32-based micro-drone), this lesson has been directly applied: every design change and piece of progress is tracked using a Requirements Traceability Matrix (RTM), combined with a linked decision-log system in Obsidian. This closes the exact gap that caused the robot failure rather than relying on a static specification document that can silently drift out of sync with the as-built system, the RTM structure forces every requirement to stay traceable to its current implementation state, while the Obsidian log preserves the reasoning behind each change. Future hardware designs should build this traceability in from day one rather than retrofitting it under deadline pressure, so that a change like the DRV8833 capacitor fix or the gripper's missing lock would be flagged as an open, trackable item not discovered as a failure on the competition floor.

Shared Ownership Over Documentation:

Having a shared specification and decision log is necessary but not sufficient, the experience showed that a document can exist, be distributed to the entire team, and still fail to prevent problems if it's treated as a fixed rulebook rather than a collaborative source of truth. Future teams should explicitly reinforce that every contributor is responsible for updating the shared documentation the moment their subsystem's real-world constraints diverge from what's written, and that doing so is not optional or someone else's job. Configuration control tools (RTM, decision logs, architecture diagrams) only work if the team's culture treats maintaining them as part of the engineering work itself not as paperwork layered on top of it.

5. Document Control Status

Initial Baseline Reference:	Frozen Archive (V1.0): original Draw.io architecture diagram, unmodified.
As-Built Delta:	Not captured in a revised diagram or BOM; documented retroactively via this post-mortem (Section 3).
Next Review Date:	N/A , project concluded. Lessons carried forward into the ESP32-based micro-drone project, where RTM and Obsidian-based decision logging are already in active use.

Appendix A. Reference Diagrams

Supporting diagrams and schematics referenced throughout this analysis, architecture diagrams, PCB routing, wiring schematics.

Figure A.1 Baseline System Architecture Diagram, see Section 2.2 (embedded above).

Figure A.2 Baseline Electrical Schematic — MCU, Driver Module & Regulator, and Power Distribution blocks. Corresponds to the power system and control software design described in Section 2.1.

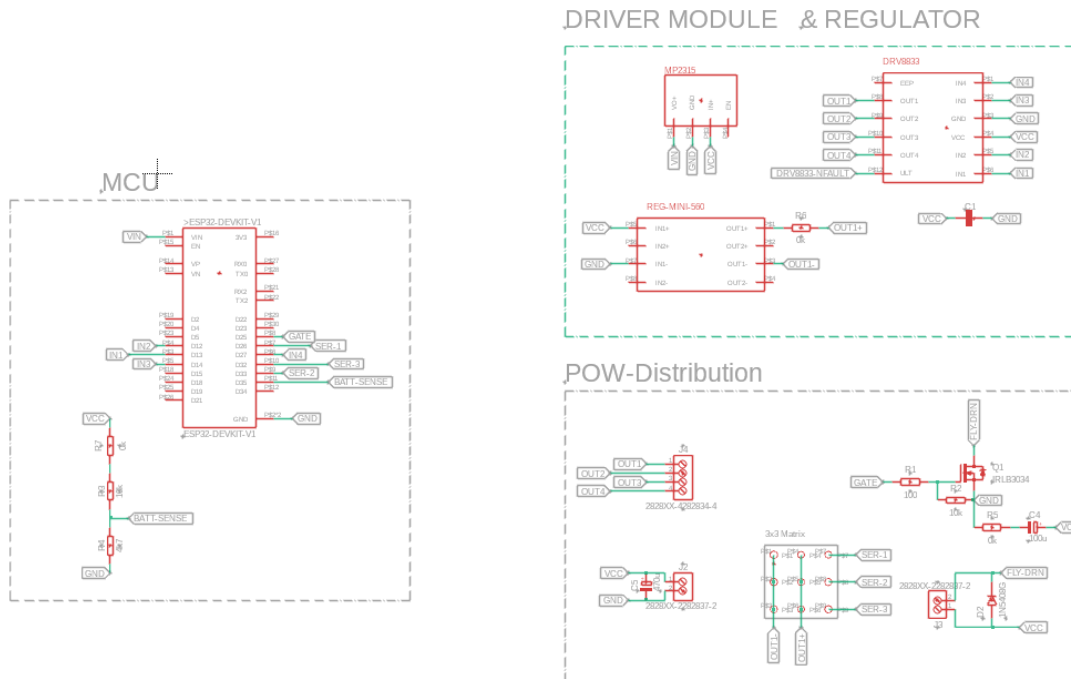
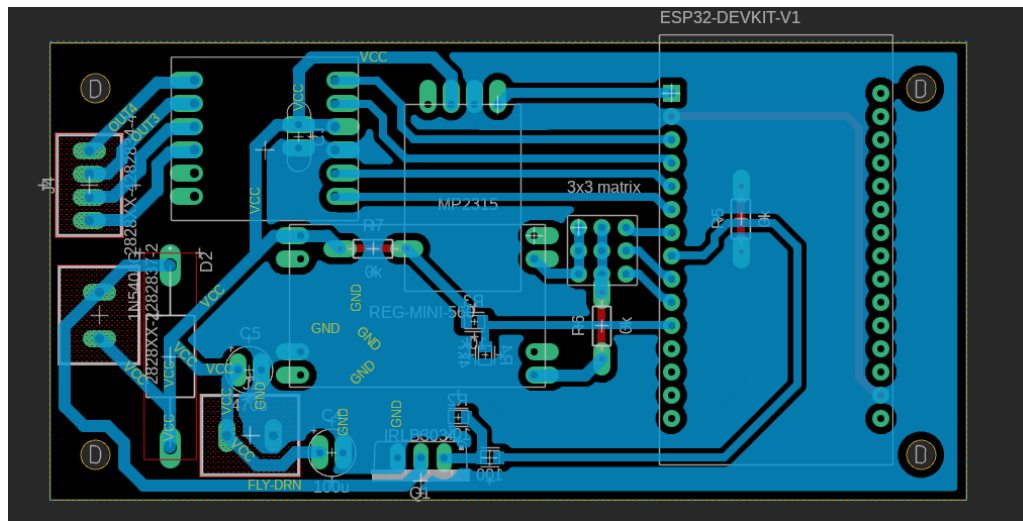


Figure A.3 Baseline PCB Routing Layout (ESP32-DEVKIT-V1). Corresponds to the power distribution and driver module design described in Section 2.1.



Note: No schematic, PCB routing, or photographic documentation exists for the Section 3 field fixes (DRV8833 capacitor addition, flywheel shaft/motor swap, or gripper joint). These changes were made under acute time

pressure, less than 24 hours before the competition, and were never logged, the absence of visual records here is itself a direct consequence of the documentation gap described in Section 3.1.

Appendix B. Supporting Media

Media B.1 First Match, working as intended before ball jammed (*forgive the loud noises*)

[\[https://drive.google.com/file/d/1scdHgbnVRL2m_vI3hFcj2EcXxqt_rOI/view?usp=drive_link\]](https://drive.google.com/file/d/1scdHgbnVRL2m_vI3hFcj2EcXxqt_rOI/view?usp=drive_link)

Media B.2 Second Match, critical failure with the arm gripper

[\[https://drive.google.com/file/d/1RFzDc48L3BI3qfyTtj-fybAQc-IiBw89/view?usp=drive_link\]](https://drive.google.com/file/d/1RFzDc48L3BI3qfyTtj-fybAQc-IiBw89/view?usp=drive_link)

Media B.3 Short test run after the match failure

[\[https://drive.google.com/file/d/1ssdyU3seLfckjGDw7xuX_gh7cEKhNHMZ/view?usp=drive_link\]](https://drive.google.com/file/d/1ssdyU3seLfckjGDw7xuX_gh7cEKhNHMZ/view?usp=drive_link)

Appendix C. Process Documentation

Process and specification documents referenced throughout this analysis, maintained as living references during the project.

Media C.1 Technical Specification Document

[\[https://docs.google.com/document/d/1-W9ElGoXg87rLVsUqR2JvMNQEONnMlI9s5iY-fWQZzo/edit?usp=sharing \]](https://docs.google.com/document/d/1-W9ElGoXg87rLVsUqR2JvMNQEONnMlI9s5iY-fWQZzo/edit?usp=sharing)